

# Utilizing NeRF-Based Rays for Spatial Perception in Fruit Counting Deduplication

1st Chong Zhou

School of Information Science and Technology,  
Beijing University of Technology  
Beijing, China  
z523598478@gmail.com

2nd Fei Xu

Hui Shou (Beijing) Technology Co.  
Beijing, China  
feixu.astro@gmail.com

3rd Ansheng Huang

School of Information Science and Technology,  
Beijing University of Technology  
Beijing, China  
ansheng@emails.bjut.edu.cn

4th Chenhuizi Wang

School of Information Science and Technology,  
Beijing University of Technology  
Beijing, China  
Wangchenhuizi@emails.bjut.edu.cn

5th Wenli Zhang\*

School of Information Science and Technology,  
Beijing University of Technology  
Beijing, China  
zhangwenli@bjut.edu.cn

**Abstract**—Accurate fruit counting is essential for effective management in orchards and plantations. Traditionally, quantifying fruits like citrus on trees involves labor-intensive manual counting. Recently, computer vision-based automated methods have emerged as a viable alternative. However, these methods often struggle with issues like occlusion by leaves and duplicate counts due to multiple appearances in the image frame. This study proposes the integration of 2D detection techniques with 3D reconstruction technologies, utilizing Rays in Neural Radiance Fields (NeRF) to overcome these challenges and enhance the precision of fruit counting on individual trees. The implementation involves targeted training of the YOLOv8 model, combined with NVIDIA's instant-ngp for swift NeRF reconstruction, and further refined by data clustering via the DBSCAN algorithm. This approach markedly increases the detection accuracy especially for a single fruit tree, resulting in a mean absolute error (MAE) of 1.5 oranges with a prediction accuracy of 90.625%, effectively refining fruit counting evaluations for a single fruit tree.

**Keywords**—Fruit Counting; YOLOv8; Neural Radiance Fields.

## I. INTRODUCTION

### A. Background and Significance

With the rapid advancements in artificial intelligence and computer vision technologies, smart agriculture has seen explosive growth, particularly in the sector of fruit cultivation. Despite the widespread use of traditional manual counting for citrus and other fruits, these methods suffer from low efficiency, high costs, and susceptibility to errors, especially under conditions of occlusion and environmental complexity. Consequently, this study combines 2D detection with 3D reconstruction techniques to propose a novel automated fruit counting method. Our method's physical significance lies in its ability to precisely map the spatial distribution of fruits on individual trees, enhancing the precision of orchard yield estimation, optimizing resource allocation, and reducing labor costs, thereby supporting high-quality agricultural development.

### B. Traditional Methods and Limitations

#### 1) Two-dimensional vision technologies

Recent advancements by researchers like Sa et al.[1], Wang et al.[2], Zhuang et al.[3], and Z. Wang et al.[4] have significantly enhanced two-dimensional vision systems for fruit count statistics, especially for fruit detection and tracking. However, 2D methods still struggle with multiple fruits on a single tree under occluded conditions due to their inability to accurately discern spatial relationships between fruits, often resulting in duplicate counts.

Conversely, three-dimensional vision models provide separate 3D coordinates for each target, enabling clear differentiation and accurate counting on individual trees, effectively addressing the shortcomings of 2D approaches.

#### 2) Three-dimensional vision technologies

Three-dimensional vision technologies like radar-assisted positioning and 3D reconstruction effectively address two-dimensional vision limitations. Techniques such as LiRaFusion and V-DETR have excelled in direct 3D object detection within spatial environments. Notably, methods based on Neural Radiance Fields (NeRF)[6] offer substantial advantages by generating high-quality 3D reconstructions and renderings, making them ideal for scenarios with complex lighting and severe occlusions. Challenges in collecting and processing extensive 3D datasets in agriculture scenarios still exists, as seen in the NeRF-RPN study by Hu et al.[7], highlight the need for integration with technologies like Region Proposal Networks (RPN) to enhance object localization and recognition. Despite its precision, NeRF-RPN demands significant computational resources, especially with high-dimensional natural orchard data.

Similarly, NeRF-Det by Xu et al.[8] refines 3D object detection using deep learning, optimizing spatial localization but also facing challenges in data collection and annotation for practical agricultural applications. To bridge the gaps of solely using 3D technologies, researchers like Liu et al.[9] are merging 2D and 3D approaches. They developed a fruit counting system using a Faster R-CNN

trained on images from mango orchards, introducing relocalization strategies with ground mark reassociation of mangos to handle occlusions, duplication and to improve counting accuracy. Additionally, methods leveraging multi-view perception and synchronized positioning systems, such as those by Jordi et al.[10] and Gan et al.[12], enhance detection reliability and efficiency. Meanwhile, Roy et al.[13] developed a custom reconstruction algorithm that significantly aids in yield estimation by providing detailed relationship data on fruit distribution.

Yet, despite recent advancements, integrating 2D and 3D vision technologies still faces challenges in accurately counting individual trees and managing repeated counts, necessitating further study into merging these technologies for fruit counting and enhanced agricultural automation.

### C. Research Focus

Our study targets citrus counting for a single fruit tree, investigating how the combination of 2D and 3D vision techniques can enhance fruit counting accuracy. The primary research focus and innovations are detailed below:

#### 1) Utilization of NeRF and NeRF-Rays:

We use instant-ngp[14] for swift NeRF training and continuous view synthesis. Then we developed functions including spatial cropping, viewpoints rearrangement and spatial perception based on the instant-ngp system. With Rays in NeRF, we successfully mapped 2D pixel information with spatial information in NeRF space.

#### 2) Integration of 2D and 3D Vision:

To gather spatial information for each citrus, we integrated two-dimensional detection with three-dimensional reconstruction from various perspectives. This hybrid approach leverages surface feature information from rendered 2D viewpoints and 3D data with rays to confirm the spatial position and independence of fruits, enhancing the fruit detection and counting workflow, data accuracy, and system efficiency. This not only boosts detection accuracy but also enhances counting reliability, making fruit count statistics more precise and practical.

#### 3) Advanced Processing and Analysis of 3D Data:

Our approach includes the use of cluster analysis for data deduplication. Spatial data is processed and analyzed further to facilitate the visualization of fruits in three-dimensional space, effectively generating spatial maps. This not only improves the accuracy of fruit count statistics but also provides a novel method for intuitively understanding the spatial arrangement and distribution of fruits. It addresses common challenges such as fruit occlusion and duplicate counts in traditional counting methods.

## II. METHODS

### A. Related Work

#### 1) NeRF and instant-ngp:

NeRF (Neural Radiance Fields) technology has gained widespread attention due to its ability to reconstruct complex three-dimensional scenes with high quality. NeRF employs a continuous function form, as described by Equation (2-1), mapping spatial positions and viewpoints to color and density values, thereby facilitating a detailed and continuous 3D visual experience.

$$V(x, \theta) = \int_{t_0}^{t_1} T(t) \sigma(t) c(t, \theta) dt \quad (2-1)$$

In this equation,  $V(x, \theta)$  represents the radiance observed from position  $x$  and viewpoint  $\theta$ , where  $\sigma$  is the density within the scene,  $c$  represents color, and  $T(t)$  is the accumulated transparency from the observer to point  $t$ .

However, traditional NeRF models face limitations in processing speed, particularly when handling large-scale datasets, where the time required for training and rendering remains a challenge. The emergence of instant-ngp, an improved implementation of NeRF, effectively addresses this challenge. By incorporating multi-resolution hash encoding techniques, instant-ngp significantly speeds up the training and rendering processes, enabling the completion of complex 3D scene modeling within seconds. This optimization not only maintains NeRF's advantages in image quality but also greatly enhances processing efficiency, demonstrating significant potential for applications requiring rapid acquisition of precise 3D information, such as in agricultural fruit tree modeling.

#### 2) YOLO

The YOLO (You Only Look Once) series of algorithms have been widely applied in various visual recognition tasks due to their cutting-edge 2D object detection capabilities. With continuous advancements, the YOLO series has undergone several iterations and updates. The version we are utilizing, YOLOv8, further optimizes the model structure and training process, enhancing the algorithm's generalization capability and detection precision. The choice to use YOLOv8 is primarily based on three points:

a) *High Precision and Efficiency*: YOLOv8 performs excellently with complex backgrounds and varying lighting conditions, accurately identifying and locating fruits.

b) *Anchor-Free Detection*: This feature allows the model to flexibly adapt to fruits of varying sizes and shapes, improving detection speed and accuracy.

c) *Optimized Network*: YOLOv8 introduces a new convolution layer and C2f modules, enhancing feature extraction and the model's robustness and adaptability.

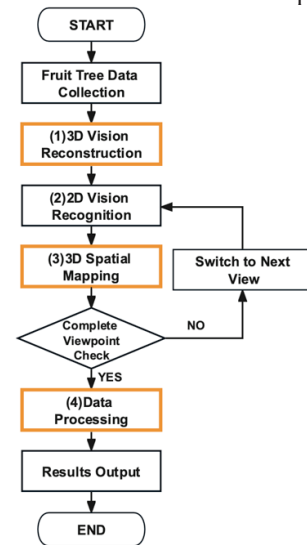


Figure 1. System Flowchart of the Method. The colored modules (1), (3), and (4) correspond to the Research Focus of the study.

## B. Implementation

Our method is presented in a systematic way, consisting of four key modules: 3D Vision Reconstruction, 2D Vision Recognition, 3D Spatial Mapping and Data Processing.

### 1) Three-dimensional Vision Reconstruction:

We employ instant-ngp for NeRF reconstruction to provide precise spatial information essential for subsequent processes, with Colmap utilized for preprocessing the captured video data. Colmap, a comprehensive photogrammetric vision software, is primarily used to obtain initial camera parameters. Initially, Colmap conducts feature extraction and matching to generate camera parameters, which are then fed into instant-ngp to construct the NeRF space. After completing the 3D reconstruction, the system employs instant-ngp for automated cropping, retaining only the area of interest with the fruit trees. The result of cropping is illustrated in Figure 2, which automatically trims based on the size of the unit cube (the central object in view). This provides a clear visual foundation for further data collection and detailed analysis.

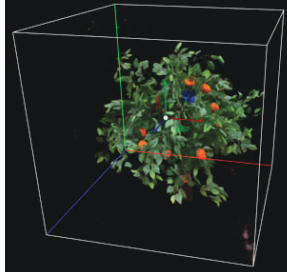


Figure 2. Diagram of automatic cropping, demonstrating the results of the automatic cropping implemented by the instant-ngp system, based on the corresponding size of the unit cube

Based on the reconstructed 3D model, this step also rearranges the viewing angles: a basic eye-level view, an oblique upward angle, and a top view. These angles are re-planned to minimize occlusions caused by leaves on the citrus fruits, aiming to observe all fruits on a single tree as much as possible, thus optimizing the effectiveness of 2D visual recognition. The re-planned angles effectively circumvent the drawbacks of original 2D video statistics and significantly reduce the requirements for capturing angles and perspectives on-site. A schematic of the viewpoint re-planning is shown in Figure 3. It demonstrates the complete path from image capture to fruit detection, where pyramids represent different camera angles, and the dashed and blue lines represent the travel trajectory. This diagram clearly illustrates how each step provides the necessary data and analytical support for precise fruit positioning, ensuring the system can effectively recognize and process visual information from complex agricultural scenes. This not only optimizes the data processing workflow but also enhances the system's adaptability and responsiveness to dynamic environments.

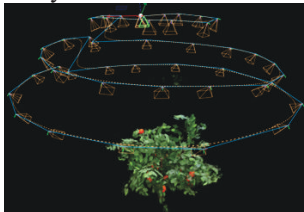


Figure 3. 2D Detection Path Diagram

### 2) Two-dimensional Vision Recognition:

The 2D vision recognition module involves targeted training of the YOLOv8 model as well as adjustments and cleaning of the dataset. The "2D Recognition Dataset" used will be detailed in the subsequent experimental section. This module is responsible for collecting screen information from the current perspective and using the model to calculate two-dimensional positional coordinates of all citrus fruits within the view. As shown in Figure 3, the system will traverse all re-planned perspectives, observing fruit trees from different angles and collecting 2D information. By training YOLOv8 on the specific dataset, the model focuses on citrus recognition on individual fruit trees and effectively distinguishes fruits from leaves.

Under common complex backgrounds and variable lighting conditions, YOLOv8 can directly predict the bounding boxes of oranges from images. Additionally, by adopting a multi-scale training strategy, the detection capability for fruits of different sizes is enhanced, and the processing of fruit tree images taken from various distances is optimized, providing a solid data foundation for subsequent 3D spatial mapping steps.

### 3) Three-dimensional Spatial Mapping:

While the 2D localization module gather pixel coordinates in different viewpoints, the 3D spatial perception module collects spatial information through a three-step implementation using rays to accurately map the fruits on a single agricultural tree. The specific implementation steps are as follows:

#### a) Rays toward Citrus:

Initially, the system utilizes the fruit bounding boxes detected by the 2D vision recognition module to obtain the corresponding 2D pixel positions. This step involves invoking the "pixel\_to\_ray\_pinhole" function in instant-ngp to simulate the pinhole camera model, transforming 2D image coordinates into 3D space rays. The pinhole camera model simplifies the imaging principles by simulating the process of light passing through a small aperture (pinhole) and forming an image on the imaging surface behind the camera. In this model, it is assumed that all light rays coming from the external world pass through this pinhole and form an inverted image on the camera's imaging surface. The pixel shift relative to the image center is identified, providing the basis for calculating the ray's direction, as all rays in the pinhole camera model are assumed to originate from the image center (the location of the pinhole). The fundamental formula is described as:

$$dir = \left( \frac{x - x_c}{f_x}, \frac{y - y_c}{f_y}, 1 \right) \quad (2-2)$$

$dir$  represents the ray direction,  $(x, y)$  is the pixel coordinates on the image plane,  $(x_c, y_c)$  is the coordinates of the image plane center, and  $f_x$  and  $f_y$  are the focal length in the horizontal and vertical directions, respectively. Detailed implementation of the function as illustrated in Algorithm1 down below. The  $uv.x$  and  $uv.y$  correspond to  $x$  and  $y$  in the formula (2-2), while  $screen\_center.x$  and  $screen\_center.y$  corresponds to  $x_c$  and  $y_c$ .

Using formula (2-2) along with the function depicted in Algorithm1, the system can accurately compute the direction of the ray in 3D space from 2D image coordinates.

---

**Algorithm 1:** pixel\_to\_ray\_pinhole: Converts pixel to ray using pinhole camera model

---

**Result:** Returns a Ray structure with origin and direction

**Data:** spp, pixel, resolution, focal.length, camera.matrix, screen.center

**Output:** Ray

Convert pixel coordinates to normalized device coordinates:  $uv \leftarrow (\text{pixel.x} / \text{resolution.x}, \text{pixel.y} / \text{resolution.y})$ ;

Calculate direction vector for the ray:  $\text{dir} \leftarrow \{;$

$(uv.x - \text{screen.center.x}) * \text{resolution.x} / \text{focal.length.x}, ;$

$(uv.y - \text{screen.center.y}) * \text{resolution.y} / \text{focal.length.y}, ;$

$1.0f ;$

$\};$

Transform the direction vector using the camera matrix:  $\text{dir} \leftarrow \text{mat3}(\text{camera.matrix}) * \text{dir}$ ;

Construct the ray with the camera position as the origin and the calculated direction:

**return** {camera.matrix[3], dir }

---

Algorithm1. Code Corresponding to the Pinhole Camera Model

The vector (ray direction) is ultimately used to determine the direction of light rays passing through specific pixel points from the camera in three-dimensional space. The rays are crucial for NeRF scene reconstruction and provide a data foundation for subsequent steps in calculating the spatial position of the oranges.

The overall implementation of the 3D spatial mapping module is shown in the following diagram:

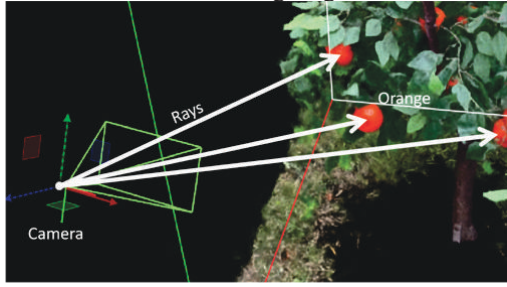


Figure 4. Overall Implementation of the 3D Spatial Module

#### b) Calculation of 3D Position:

Subsequently, the system combines the depth information within instant-ngp and calculates along the direction of the ray propagation. By incrementally increasing the depth value, the system determines the specific location of the ray in 3D space, thus computing the three-dimensional coordinates of the fruit from the current viewpoint. The calculation formula for the citrus coordinates is:

$$\text{Orange}(x, y, z) = \text{Camera}(x, y, z) + \text{Depth} \times \text{Ray}(x, y, z) \quad (2-3)$$

Where  $\text{Orange}(x, y, z)$  represents the unknown coordinates of the orange,  $\text{Camera}(x, y, z)$  the known current camera coordinates,  $\text{Depth}$  the depth from the camera to the fruit surface pixels, and  $\text{Ray}(x, y, z)$  the ray sent from the camera to the orange.

#### c) Incremental Processing of 3D Information:

To further enhance the positioning accuracy, the system meticulously adjusts the collected 3D information through fine step increments. In this system, the step value is set to 0.03, representing the average radius of an orange in NeRF space. By setting this specific step value, the system finely simulates the pixel coordinates corresponding to the orange from surface to core. The formula for calculating the stepped coordinates of the orange is analogous to (2-3), as follows:

$$\begin{aligned} \text{Adjusted\_Orange}(x, y, z) \\ = \text{Orange}(x, y, z) + 0.03 \times \text{Ray}(x, y, z) \end{aligned} \quad (2-4)$$

Where  $\text{Adjusted\_Orange}(x, y, z)$  represents the orange coordinates after adjustment, and  $\text{Ray}(x, y, z)$  is the ray sent from the camera to the orange. This fine-grained step adjustment helps avoid data errors caused by changes in perspective, especially in cases of multi-angle shooting. By precisely controlling the additional movement distance from the surface to the core, the system ensures that the position of the fruit observed from any angle is accurately mapped, thereby significantly reducing errors and potential duplicate counts.

#### 4) Data Processing and Analysis:

In this module, the system uses the DBSCAN clustering algorithm to process the 3D data obtained from the spatial mapping module. Density-Based Spatial Clustering of Applications with Noise is a density-based clustering algorithm that can identify clusters of any shape while effectively distinguishing and excluding noise points.

Key parameters of DBSCAN — neighborhood size ( $\text{eps}$ ) and minimum points ( $\text{minPoints}$ ) — are carefully adjusted to optimize the clustering results. By setting the  $\text{eps}$  value to 0.03 (the radius of an orange) and the  $\text{minPoints}$  value to 3 (approximate number of observations per orange), the algorithm better defines the clustering boundaries of the fruits. The working principle of the DBSCAN clustering algorithm can be summarized as follows:

$$\text{eps\_neighborhood} = \{p' \in D \mid \text{dist}(p, p') \leq \text{eps}\} \quad (2-5)$$

$$\text{Core\_Point} = p \in D \mid |\text{eps\_neighborhood}(p)| \geq \text{minPoints} \quad (2-6)$$

Here,  $\text{eps\_neighborhood}$  represents the neighborhood of a point in the dataset, i.e., all points within a specified neighborhood size ( $\text{eps}$ ) distance;  $\text{Core\_Point}$  represents a core point, which must have a number of points in its neighborhood not less than the specified minimum points ( $\text{minPoints}$ ). In the formula,  $D$  represents the dataset,  $p$  and  $p'$  represent any two points in the dataset, and  $\text{dist}(p, p')$  represents the distance between  $p$  and  $p'$ .

To further analyze and verify the clustering results, the data processing analysis module also includes a visualization processing step, which will be presented as part of the result in experiment part.

### III. EXPERIMENT

#### A. Introduction of datasets

##### 1) 2D Recognition Dataset:

The two-dimensional recognition dataset primarily serves to train an appropriate model for the 2D vision recognition module. To meet the specific requirements of citrus planting scenarios, data preprocessing and augmentation strategies include:

a) *Automatic Orientation Adjustment:* Ensures all images have the correct orientation before entering the model to maintain data consistency.

b) *Resizing:* All images are uniformly resized to 1280x1280 pixels, using the reflection padding method to fit the new size.

c) *Data Augmentation:* Each training sample generates three output variants by adjusting saturation (-40% to +40%) and exposure (-20% to +20%) to enhance the model's adaptability to lighting variations.

d) *Dataset Cleaning*: Adjustments are made to the dataset annotations to avoid selecting leaves, enabling the trained model to exclude parts containing non-fruit elements (such as leaves) during inference.

The optimized dataset after processing contains multi-scenario images collected from orchards, totaling 4800 images for the training set, 400 for the validation set, and 150 for the test set, ensuring that the model performs with high accuracy and robustness in various orchard scenarios.

## 2) 3D Reconstruction Dataset

Another dataset used in this study was collected from a single simulated citrus tree with varying fruit distributions, called 3D Reconstruction Dataset. Each data involved changing the distribution of citrus fruits before the video capture to test the method's effectiveness in fruit counting. The collection device used a single-lens camera, with the specific collection method involving oblique and top-down surround shooting at a constant speed to ensure data consistency and comparability. To ensure effective testing and three-dimensional reconstruction, ten sets of fruit tree video information with different fruit distributions were collected for three-dimensional reconstruction and quantity detection. In Figure 5, the left image shows a viewpoint from the three-dimensional reconstruction dataset, and the right image shows the same viewpoint after reconstruction. The comparison illustrates the high-quality reconstruction results of instant-ngp and the collection effectiveness of the three-dimensional reconstruction dataset.



Figure 5. Comparison from the Same Viewpoint between original dataset and instant-ngp reconstruction

## B. Evaluation Metrics

To validate the system's effectiveness in counting fruits on a single tree, several accuracy metrics were introduced for evaluation. The formulas for these metrics are detailed below, where  $y_i$  represents the actual number of fruits on the tree counted manually,  $\hat{y}_i$  is the number of fruits predicted by the system, and  $n$  is the number of samples in the dataset.

1) *Mean Absolute Error (MAE)* measures the average absolute difference between predicted values and actual values:

$$MAE = \left(\frac{1}{n}\right) \sum |y_i - \hat{y}_i| \quad (3-1)$$

2) *Mean Squared Error (MSE)* measures the average of the squares of the differences between predicted and actual values:

$$MSE = (1/n) \sum (y_i - \hat{y}_i)^2 \quad (3-2)$$

3) *Mean Absolute Percentage Error (MAPE)* measures the percentage of prediction errors relative to actual values:

$$MAPE = (100\%/n) \sum |(y_i - \hat{y}_i) / y_i| \quad (3-3)$$

4) *Estimation Accuracies* measure how closely predicted values match actual values, typically expressed as a percentage:

$$Estimation\ Accuracies = (1 - (\sum |y_i - \hat{y}_i|) / (\sum y_i)) \quad (3-4)$$

## C. Experiment and Results

Our study uses a 3D reconstruction dataset to test our method (YOLOv8+NeRF). The system uses rendered videos reconstructed from this dataset to conduct comparative experiments with current common 2D detection and tracking methods. Conducting 2D detection and tracking experiments directly with the original 3D visual reconstruction dataset would lead to particularly severe issues with duplicate counting due to multiple surrounding captures of the fruit trees, losing comparability. Thus, the study planned the viewing angles for the reconstructed model to encompass a 45-degree top-down surround view and exported the video, as shown in Figure 6. A total of ten fruit trees with different fruit distributions formed a fruit tree test dataset corresponding to the 3D reconstruction dataset, used to compare the effectiveness of the YOLOv8+NeRF method within this system, including conducting comparative experiments with two commonly used and efficient two-dimensional object detection and tracking methods, YOLOv8+Botsort and YOLOv8+Bytetrack, with experiment environments in Table I. and results summarized in Table II.

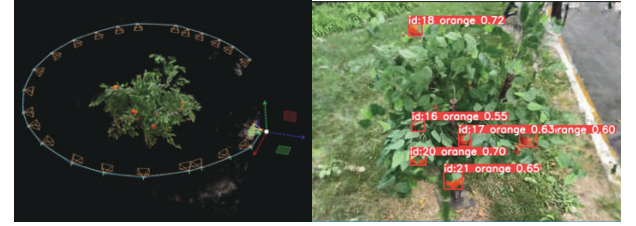


Figure 6. Viewpoints for rendering & YOLOv8+Bytetrack test result

On the left side of Figure 6, pyramids represent different camera angles with dashed and blue lines marking the trajectory of the viewpoints. To minimize duplicate counting errors common in 2D object detection and tracking methods, the rendering path avoids overlapping of starting and ending viewpoints. On the right side, the image presents a single count result from the YOLOv8+Bytetrack detection applied to the fruit tree test dataset. The orange boxes highlight the detected oranges, with numbers indicating confidence levels and IDs representing the tracked trajectories of the oranges.

TABLE I. ENVIRONMENTS

| Hardware Environment           | Software Environment            |
|--------------------------------|---------------------------------|
| CPU: AMD Ryzen 7800X3D         | Tools: VS Code, instant-ngp     |
| GPU: NVIDIA GeForce RTX 4070Ti | Language: Python 3.9, CUDA 11.5 |

TABLE II. RESULTS

| Dataset                   | Methods            | Scores |      |        |            |
|---------------------------|--------------------|--------|------|--------|------------|
|                           |                    | MAE    | MSE  | MAPE   | Accuracies |
| Fruit Tree Test Dataset   | YOLOv8+Botsort     | 4.4    | 20.4 | 27.5%  | 72.5%      |
| Fruit Tree Test Dataset   | YOLOv8+Bytetrack   | 3.6    | 14   | 22.5%  | 77.5%      |
| 3D reconstruction Dataset | <b>YOLOv8+NeRF</b> | 1.5    | 2.5  | 9.375% | 90.625%    |

In Table II. Results, methods have been tested under different datasets and methods to evaluate performance of fruit counting. The results show that YOLOv8+NeRF excelled in all the metrics and an accuracy rate as high as 90.625%. This indicates that our method, using the same model for inference as the other two methods, allows the system's modules to collaborate more effectively in achieving more precise statistics on the number of fruits on individual trees. The proposed model performs better than comparison models because it uses the unique 3D coordinate mapping to accurately identify and count each fruit. Traditional methods only rely on object trajectory tracking, which is prone to duplication errors.

The system not only provides the number of fruits as an output result but also offers visual results obtained from the data processing analysis module. Figure below shows the visualization results, which also represent the system's spatial accuracy. It provides researchers with an intuitive understanding of fruit distribution data that is unobstructed.

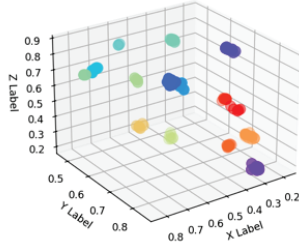


Figure 7. YOLOv8+NeRF Visualization Results Display. The XYZ axes represent the normalized coordinate system in NeRF space, and different colored points in the figure represent each clustered citrus fruit.

#### IV. CONCLUSION AND FUTURE WORK

##### A. Conclusion

This study successfully developed a spatial localization method that combines NeRF with two-dimensional vision recognition technologies for high-precision detection and counting of fruits on individual trees. The system effectively resolves issues of fruit occlusion and duplicate counting prevalent in traditional methods. Experimental results demonstrate that the system exhibits good adaptability and accuracy on fruit trees with varying fruit distributions.

##### B. Future Work

Future research will broaden the capabilities of this system. By switching the YOLOv8 model weights, the system will better adapt to various fruits like apples, pears, and apricots, ensuring accurate counts across different tree species. Moreover, we plan to incorporate advanced 3D reconstruction technologies such as dynamic NeRF for

tracking moving or growing fruits, thus enhancing the model's dynamic capturing capabilities and real-time processing efficiency. This technology's application is also expected to extend into urban IoT, enhancing numerous smart cities localizing applications.

In conclusion, the successful implementation of this study introduces a new dimension to future agricultural technologies, illustrating the vast potential of merging deep learning and NeRF modeling in both agriculture and smart city contexts. With ongoing technological enhancements and application expansions, we anticipate a broader implementation of agricultural automation and intelligence.

#### ACKNOWLEDGMENT

This study was supported by the National Natural Science Foundation of China (NSFC) Program 62276009.

#### REFERENCES

- [1] I. Sa, Z. Ge, F. Dayoub, B. Upcroft, T. Perez, and C. McCool, "Deepfruits: A fruit detection system using deep neural networks," *Sensors*, vol. 16, no. 8, 1222, 2016.
- [2] J. Wang and W. Zhang, "Research and implementation of fruit detection tracking system based on deep learning," *Master's Electronics*, vol. 2023, no. 1, 2023.
- [3] H. Zhuang, J. Zhou, Y. Lin, H. Peng, and H. Lin, "Research on citrus fruit identification and counting based on improved YOLOv5+DeepSort," *Southern Agricultural Machinery*, 2023.
- [4] Z. Wang, K. Walsh, and A. Koirala, "Mango fruit load estimation using a video-based MangoYOLO—Kalman Filter—Hungarian Algorithm method," *Sensors*, vol. 19, no. 6, 2742, 2019.
- [5] H. Gan, W. S. Lee, V. Alchanatis, and A. Abd-Elrahman, "Active thermal imaging for immature citrus fruit detection," *Biosystems Engineering*, vol. 198, pp. 291-303, 2020.
- [6] B. Ben Mildenhall, Pratul P. Srinivasan, Matthew Tancik, Jonathan T. Barron, Ravi Ramamoorthi, and Ren. "NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis," *ECCV*, pp. 405-421, 2020.
- [7] B. Hu, J. Huang, Y. Liu, Y.-W. Tai, and C.-K. Tang, "NeRF-RPN: A general framework for object detection in NeRFs," *Computer Vision and Pattern Recognition*, arXiv:2211.11646, 2023.
- [8] C. Xu, B. Wu, J. Hou, S. Tsai, R. Li, J. Wang, W. Zhan, Z. He, P. Vajda, K. Keutzer, and M. Tomizuka, "NeRF-Det: Learning geometry-aware volumetric representation for multi-view 3D object detection," *Computer Vision and Pattern Recognition*, pp. 23320-23330, 2023.
- [9] X. Liu, S. W. Chen, C. Liu, S. S. Shivakumar, J. Das, C. J. Taylor, J. Underwood, and V. Kumar, "Monocular camera based fruit counting and mapping with semantic data association," *IEEE Robotics and Automation Letters*, vol. 4, no. 3, pp. 2296-2303, doi:10.1109/LRA.2019.2901987, 2019.
- [10] J. Gené-Mola, R. Sanz-Cortellá, J. R. Rosell-Polo, et al., "Fruit detection and 3D location using instance segmentation neural networks and structure-from-motion photogrammetry," *Computers and Electronics in Agriculture*, vol. 169, 105165, 2020.
- [11] M. Stein, S. Bargoti, and J. Underwood, "Image based mango fruit detection, localisation and yield estimation using multiple view geometry," *Sensors*, vol. 16, no. 11, 1915, 2016.
- [12] H. Gan, W. S. Lee, and V. Alchanatis, "A prototype of an immature citrus fruit yield mapping system," *2017 ASABE Annual International Meeting*. American Society of Agricultural and Biological Engineers, 2017.
- [13] P. Roy, W. Dong, and V. Isler, "Registering reconstructions of the two sides of fruit tree rows," *2018 International Conference on Intelligent Robots and Systems (IROS)*, pp. 1-9, 2018.
- [14] A. Tewari, M. Zollhöfer, J. Thies, C. Richardt, and C. Theobalt, "Instant neural graphics primitives with a multiresolution hash encoding," *ACM Trans. Graph.*, 41(4), 7 2022.